

MATH GR 5320 Homework VI

```
In [69]: import warnings
warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd
from scipy import stats
import matplotlib.pyplot as plt
from matplotlib import rcParams

CU = dict(
    navy      = '#003087',
    blue      = '#75B2DD',
    dark      = '#001a4d',
    gold      = '#c8a951',
    slate     = '#4a6fa5',
    warm_white = '#f7f9fc',
    grid      = '#dde4ef',
    text      = '#1a1a2e',
)

rcParams.update({
    'figure.facecolor' : CU['warm_white'],
    'axes.facecolor'   : CU['warm_white'],
    'axes.edgecolor'   : CU['navy'],
    'axes.labelcolor'  : CU['text'],
    'axes.labelweight' : 'bold',
    'axes.titleweight' : 'bold',
    'axes.titlecolor'  : CU['navy'],
    'axes.spines.top'  : False,
    'axes.spines.right': False,
    'axes.grid'        : True,
    'grid.color'        : CU['grid'],
    'grid.linestyle'    : '--',
    'grid.linewidth'    : 0.6,
    'lines.linewidth'   : 1.8,
    'xtick.color'       : CU['text'],
    'ytick.color'       : CU['text'],
    'font.family'       : 'serif',
    'font.serif'        : ['Palatino', 'Georgia', 'Times New Roman', 'DejaVu'],
    'figure.dpi'        : 120,
    'legend.framealpha' : 0.9,
    'legend.edgecolor'  : CU['grid'],
})

def cu_stamp(ax, label='MATH GR 5320'):
    ax.text(0.99, 0.01, label, transform=ax.transAxes,
           fontsize=7, color=CU['blue'], alpha=0.5,
           ha='right', va='bottom', fontfamily='monospace')

def load_bloomberg(filename):
    df = pd.read_csv(
        f'data/{filename}',
        parse_dates=['Dates'],
        na_values=['#N/A N/A', '#N/A', 'N/A', ''])
```

```

    )
    df = df.sort_values('Dates').set_index('Dates')
    df = df.apply(pd.to_numeric, errors='coerce')
    return df.dropna(subset=['PX_LAST'])

aapl = load_bloomberg('AAPL-bloomberg.csv')
cat = load_bloomberg('CAT-bloomberg.csv')

print(f'✅ AAPL: {len(aapl):,} rows | {aapl.index.min().date()} → {aapl.index.max().date()}')
print(f'✅ CAT: {len(cat):,} rows | {cat.index.min().date()} → {cat.index.max().date()}')

✅ AAPL: 10,898 rows | 1982-11-15 → 2026-02-11
✅ CAT: 11,480 rows | 1980-07-28 → 2026-02-11

```

Q1 Portfolio Mean ; Variance

```

In [70]: PURCHASE_DATE = '1997-10-13'
        SHARES_AAPL = 24679
        SHARES_CAT = 171

        # Align both series
        common_idx = aapl.index.intersection(cat.index)
        common_idx = common_idx[common_idx >= PURCHASE_DATE]

        px_aapl = aapl.loc[common_idx, 'PX_LAST']
        px_cat = cat.loc[common_idx, 'PX_LAST']

        # Portfolio value  $S(t) = \text{shares}_A * P_A(t) + \text{shares}_C * P_C(t)$ 
        S = SHARES_AAPL * px_aapl + SHARES_CAT * px_cat
        S.name = 'Portfolio_S'

        # GBM log returns:  $r(t) = \log(S(t) / S(t-1))$ 
        log_ret = np.log(S / S.shift(1)).dropna()

        print(f'Portfolio value on {PURCHASE_DATE}: ${S.iloc[0]:,.2f}')
        print(f'Portfolio value (latest): ${S.iloc[-1]:,.2f}')
        print(f'Total log-return observations: {len(log_ret):,}')

```

Portfolio value on 1997-10-13: \$10,113.94
 Portfolio value (latest): \$6,931,589.50
 Total log-return observations: 7,126

```

In [71]: # — Q1 · Window-Based Mean & Variance (2yr, 5yr, 10yr) —————
        TRADING_DAYS = 252

        windows = {
            '2yr' : 2 * TRADING_DAYS,
            '5yr' : 5 * TRADING_DAYS,
            '10yr' : 10 * TRADING_DAYS,
        }

        # Daily estimates – annualise by *252 (mean) and *252 (variance)
        win_mean = {}
        win_var = {}
        for label, w in windows.items():
            win_mean[label] = log_ret.rolling(w).mean() * TRADING_DAYS
            win_var[label] = log_ret.rolling(w).var() * TRADING_DAYS

        print('Window-based estimates (latest values):')
        print(f' {"Window":<8} {"Mean (ann)":>12} {"Var (ann)":>12}')

```

```
print(' ' + '-'*36)
for label in windows:
    m = win_mean[label].iloc[-1]
    v = win_var[label].iloc[-1]
    print(f' {label:<8} {m:>12.6f} {v:>12.6f}')
```

Window-based estimates (latest values):

Window	Mean (ann)	Var (ann)
2yr	0.191105	0.075996
5yr	0.141090	0.074591
10yr	0.243816	0.081861

```
In [72]: lambdas = {
    '2yr (λ=0.9972531953)' : 0.9972531953,
    '5yr (λ=0.9989003714)' : 0.9989003714,
    '10yr (λ=0.9994500345)' : 0.9994500345,
}

def ewma_mean_var(series, lam):
    """
    Exact EWMA mean and variance via recurrence (Lecture 4, Slide 48).

     $m_N = (1-\lambda) \cdot a_N + \lambda \cdot m_{N-1}$  ← mean recurrence
     $r_N = (1-\lambda) \cdot a_N^2 + \lambda \cdot r_{N-1}$  ← 2nd moment recurrence
     $v_N = r_N - m_N^2$  ← variance
    """
    alpha = 1 - lam
    a = series.values
    m = np.zeros(len(a))
    r = np.zeros(len(a))
    m[0] = a[0]
    r[0] = a[0]**2
    for i in range(1, len(a)):
        m[i] = alpha * a[i] + lam * m[i-1]
        r[i] = alpha * a[i]**2 + lam * r[i-1]
    v = r - m**2
    return (
        pd.Series(m * TRADING_DAYS, index=series.index),
        pd.Series(v * TRADING_DAYS, index=series.index),
    )

ew_mean = {}
ew_var = {}
for label, lam in lambdas.items():
    ew_mean[label], ew_var[label] = ewma_mean_var(log_ret, lam)

print('EWMA estimates (latest values):')
print(f' {"Label":<30} {"Mean (ann)":>12} {"Var (ann)":>12}')
print(' ' + '-'*58)
for label in lambdas:
    m = ew_mean[label].iloc[-1]
    v = ew_var[label].iloc[-1]
    print(f' {label:<30} {m:>12.6f} {v:>12.6f}')
```

EWMA estimates (latest values):

Label	Mean (ann)	Var (ann)
2yr ($\lambda=0.9972531953$)	0.206284	0.074425
5yr ($\lambda=0.9989003714$)	0.208544	0.081316
10yr ($\lambda=0.9994500345$)	0.255185	0.088162

```
In [73]: win_colors = [CU['navy'], CU['slate'], CU['blue']]
ew_colors = [CU['gold'], '#e07b39', '#b84c1a']
win_labels = list(windows.keys())
ew_labels = list(lambdas.keys())

fig, axes = plt.subplots(2, 2, figsize=(15, 9))
fig.patch.set_facecolor(CU['warm_white'])
fig.suptitle('Q1 – Portfolio Mean & Variance: Window vs EWMA | MATH GR
             fontsize=13, fontweight='bold', color=CU['navy'])

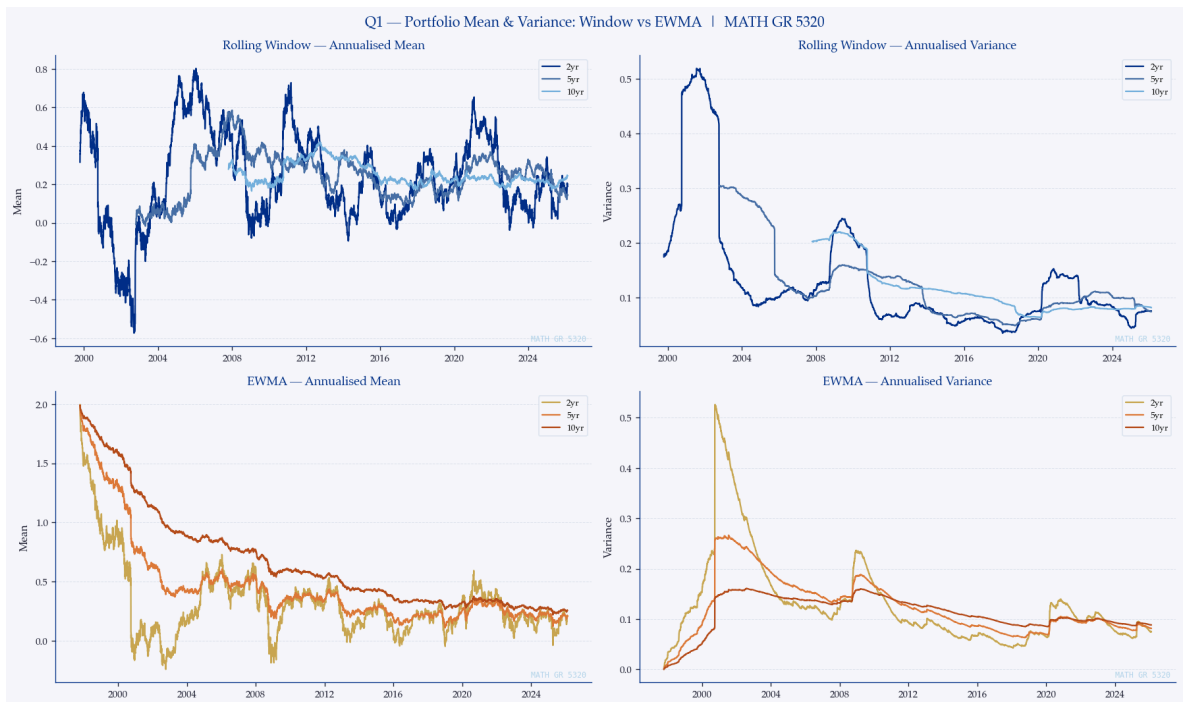
ax = axes[0, 0]
for label, color in zip(win_labels, win_colors):
    ax.plot(win_mean[label].index, win_mean[label], color=color, lw=1.4,
ax.set_title('Rolling Window – Annualised Mean')
ax.set_ylabel('Mean')
ax.legend(fontsize=8)
cu_stamp(ax)

ax = axes[0, 1]
for label, color in zip(win_labels, win_colors):
    ax.plot(win_var[label].index, win_var[label], color=color, lw=1.4, la
ax.set_title('Rolling Window – Annualised Variance')
ax.set_ylabel('Variance')
ax.legend(fontsize=8)
cu_stamp(ax)

ax = axes[1, 0]
for label, color in zip(ew_labels, ew_colors):
    ax.plot(ew_mean[label].index, ew_mean[label], color=color, lw=1.4,
            label=label.split('(')[0].strip())
ax.set_title('EWMA – Annualised Mean')
ax.set_ylabel('Mean')
ax.legend(fontsize=8)
cu_stamp(ax)

ax = axes[1, 1]
for label, color in zip(ew_labels, ew_colors):
    ax.plot(ew_var[label].index, ew_var[label], color=color, lw=1.4,
            label=label.split('(')[0].strip())
ax.set_title('EWMA – Annualised Variance')
ax.set_ylabel('Variance')
ax.legend(fontsize=8)
cu_stamp(ax)

plt.tight_layout()
plt.show()
```



```
In [74]: fig, axes = plt.subplots(1, 2, figsize=(15, 5))
fig.patch.set_facecolor(CU['warm_white'])
fig.suptitle('Q1 — Window vs EWMA Comparison | MATH GR 5320',
             fontsize=13, fontweight='bold', color=CU['navy'])

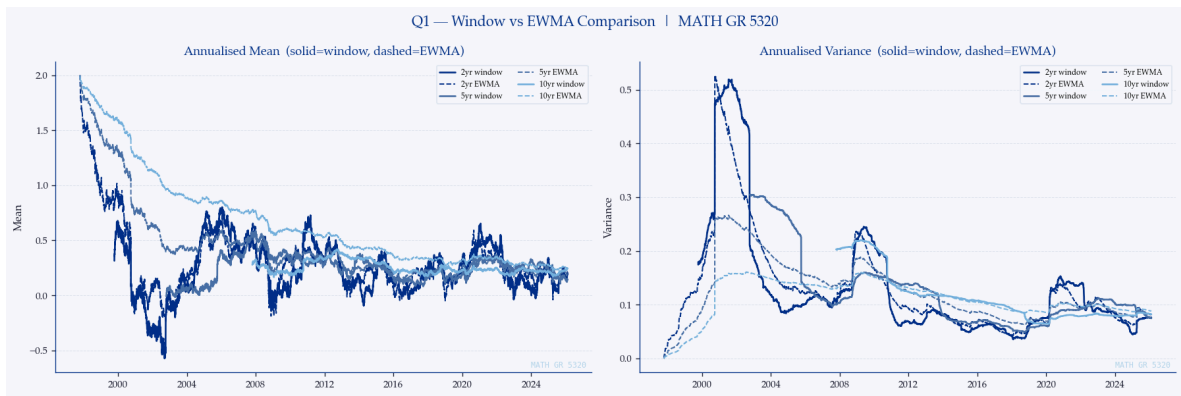
pairs = [
    ('2yr', '2yr ( $\lambda=0.9972531953$ )'),
    ('5yr', '5yr ( $\lambda=0.9989003714$ )'),
    ('10yr', '10yr ( $\lambda=0.9994500345$ )'),
]
colors = [CU['navy'], CU['slate'], CU['blue']]

for (wlabel, elabel), color in zip(pairs, colors):
    axes[0].plot(win_mean[wlabel], color=color, lw=1.6, ls='-', label=f
    axes[0].plot(ew_mean[elabel], color=color, lw=1.2, ls='--', label=f
    axes[1].plot(win_var[wlabel], color=color, lw=1.6, ls='-', label=f
    axes[1].plot(ew_var[elabel], color=color, lw=1.2, ls='--', label=f

axes[0].set_title('Annualised Mean (solid=window, dashed=EWMA)')
axes[0].set_ylabel('Mean')
axes[0].legend(fontsize=7, ncol=2)
cu_stamp(axes[0])

axes[1].set_title('Annualised Variance (solid=window, dashed=EWMA)')
axes[1].set_ylabel('Variance')
axes[1].legend(fontsize=7, ncol=2)
cu_stamp(axes[1])

plt.tight_layout()
plt.show()
```



Portfolio VaR & ES

```
In [ ]: CONF_VAR = 0.99
CONF_ES = 0.975
H = 5
T = H / TRADING_DAYS
WINDOW = 2 * TRADING_DAYS

z_var = stats.norm.ppf(1 - CONF_VAR)
alpha_es = 1 - CONF_ES
z_es = stats.norm.ppf(alpha_es)

mu_ann = win_mean['2yr']
var_ann = win_var['2yr']
sig_ann = np.sqrt(var_ann)

sig_h = sig_ann * np.sqrt(T)
drift = (mu_ann - 0.5 * var_ann) * T

# VaR: invert lognormal CDF at (1-p) quantile
gbm_VaR = S * (1 - np.exp(sig_h * z_var + drift))

# ES: E[loss | loss > VaR] via Black-Scholes integral (W5 solution sheet)
gbm_ES = S * (1 - np.exp(drift + 0.5 * var_ann * T)
              * stats.norm.cdf(z_var - sig_h) / (1 - CONF_VAR))

ret_a = np.log(px_aapl / px_aapl.shift(1)).dropna()
ret_c = np.log(px_cat / px_cat.shift(1)).dropna()
common = ret_a.index.intersection(ret_c.index)
ret_a, ret_c = ret_a.loc[common], ret_c.loc[common]

val_a = SHARES_AAPL * px_aapl.loc[common]
val_c = SHARES_CAT * px_cat.loc[common]
S_2 = val_a + val_c
w_a, w_c = val_a / S_2, val_c / S_2

mu_a = ret_a.rolling(WINDOW).mean() * TRADING_DAYS
mu_c = ret_c.rolling(WINDOW).mean() * TRADING_DAYS
var_a = ret_a.rolling(WINDOW).var() * TRADING_DAYS
var_c = ret_c.rolling(WINDOW).var() * TRADING_DAYS
cov_ac = ret_a.rolling(WINDOW).cov(ret_c) * TRADING_DAYS

# BM driver correlation  $p(dW_A, dW_C) = p(\log\text{-}ret_A, \log\text{-}ret_C)$ 
corr_ac = ret_a.rolling(WINDOW).corr(ret_c)

mu_p_2 = w_a * mu_a + w_c * mu_c
```

```

var_p_2 = w_a**2 * var_a + w_c**2 * var_c + 2 * w_a * w_c * cov_ac
sig_p_2 = np.sqrt(var_p_2)

mu_p_h = mu_p_2 / TRADING_DAYS * H
sig_p_h = sig_p_2 / np.sqrt(TRADING_DAYS) * np.sqrt(H)

norm_VaR = -(mu_p_h + z_var * sig_p_h) * S_2
norm_ES = -(mu_p_h - sig_p_h * stats.norm.pdf(z_es) / alpha_es) * S_2

print('=' * 55)
print(' Q2 · VaR & ES – Latest Values')
print('=' * 55)
print(f' Method 1 – GBM (exact lognormal, Q1  $\mu/\sigma^2$ ):')
print(f' VaR (99%, 5d) : ${gbm_VaR.dropna().iloc[-1]:>12,.0f}')
print(f' ES (97.5%) : ${gbm_ES.dropna().iloc[-1]:>12,.0f}')
print()
print(f' Method 2 – Two-Stock Normal Portfolio:')
print(f' VaR (99%, 5d) : ${norm_VaR.dropna().iloc[-1]:>12,.0f}')
print(f' ES (97.5%) : ${norm_ES.dropna().iloc[-1]:>12,.0f}')
print()
print(f' BM Driver Corr  $\rho(dW_A, dW_C)$  : {corr_ac.dropna().iloc[-1]:.4f}')

```

Q2 · VaR & ES – Latest Values

Method 1 – GBM (exact lognormal, Q1 μ/σ^2):

VaR (99%, 5d) : \$ 579,445
 ES (97.5%) : \$ 662,028

Method 2 – Two-Stock Normal Portfolio:

VaR (99%, 5d) : \$ 597,001
 ES (97.5%) : \$ 600,071

BM Driver Corr $\rho(dW_A, dW_C)$: 0.3507

```

In [77]: # — Q2 · Plot: VaR, ES, and BM Driver Correlation —————
from matplotlib.ticker import FuncFormatter
dollar_fmt = FuncFormatter(lambda x, _: f'${x/1e6:.1f}M' if x >= 1e6 else

fig, axes = plt.subplots(3, 1, figsize=(14, 12), sharex=True)
fig.patch.set_facecolor(CU['warm_white'])
fig.suptitle('Q2 – 99% 5-day VaR & 97.5% ES | MATH GR 5320',
             fontsize=13, fontweight='bold', color=CU['navy'])

axes[0].plot(gbm_VaR.dropna(), color=CU['navy'], lw=1.6, ls='-', label=
axes[0].plot(norm_VaR.dropna(), color=CU['gold'], lw=1.4, ls='--', label=
axes[0].set_title('VaR (99%, 5-day) – GBM Exact vs Two-Stock Normal')
axes[0].set_ylabel('VaR')
axes[0].yaxis.set_major_formatter(dollar_fmt)
axes[0].legend(fontsize=9)
cu_stamp(axes[0])

axes[1].plot(gbm_ES.dropna(), color=CU['navy'], lw=1.6, ls='-', label=
axes[1].plot(norm_ES.dropna(), color=CU['gold'], lw=1.4, ls='--', label=
axes[1].set_title('ES (97.5%) – GBM Exact vs Two-Stock Normal')
axes[1].set_ylabel('ES')
axes[1].yaxis.set_major_formatter(dollar_fmt)
axes[1].legend(fontsize=9)
cu_stamp(axes[1])

```

```

axes[2].plot(corr_ac.dropna(), color=CU['slate'], lw=1.4)
axes[2].axhline(corr_ac.dropna().mean(), color=CU['gold'], lw=1.0, ls='--',
                label=f'Mean  $\rho(dW_A, dW_C) = \{corr\_ac.dropna().mean():.3f\}$ ')
axes[2].set_title('BM Driver Correlation  $\rho(dW_A, dW_C) = \rho(\log-ret_A, \log-ret_C)$ ')
axes[2].set_ylabel('Correlation')
axes[2].legend(fontsize=9)
cu_stamp(axes[2])

plt.tight_layout()
plt.show()

```



Reduced form modelling

```

In [78]: # — Q3 .
# Constant hazard rate  $\lambda$  — exponential distribution
#  $s(t) = P(\tau > t) = e^{-\lambda t}$ 
#  $P(\tau \leq t) = 1 - e^{-\lambda t}$ 
#  $P(t1 < \tau \leq t2) = s(t1) - s(t2) = e^{-\lambda t1} - e^{-\lambda t2}$ 

lam = 0.0074

def survival(t, lam=lam):
    return np.exp(-lam * t)

def prob_default_by(t, lam=lam):
    """ $P(\tau \leq t)$  — cumulative default probability"""
    return 1 - survival(t, lam)

def prob_default_between(t1, t2, lam=lam):

```



```

    """P( $t_1 < \tau \leq t_2$ )"""
    return survival(t1, lam) - survival(t2, lam)

pa = prob_default_by(5)
pb = prob_default_between(3, 4)

print('=' * 55)
print(f'  Q3 · Reduced Form Model    $\lambda = \{\text{lam}\}$ ')
print('=' * 55)
print(f'  (a) P(default within 5 years)           = {pa:.6f}   ({pa*100:.4f})')
print(f'        Formula:  $1 - e^{(-\lambda \cdot 5)} = 1 - e^{(-\{\text{lam} \cdot 5\})}$ ')
print()
print(f'  (b) P(default between 3 and 4 years) = {pb:.6f}   ({pb*100:.4f})')
print(f'        Formula:  $e^{(-\lambda \cdot 3)} - e^{(-\lambda \cdot 4)}$ ')
print(f'                =  $e^{(-\{\text{lam} \cdot 3\})} - e^{(-\{\text{lam} \cdot 4\})}$ ')
print(f'                = {survival(3):.6f} - {survival(4):.6f}')

t = np.linspace(0, 20, 500)

fig, axes = plt.subplots(1, 2, figsize=(13, 5))
fig.patch.set_facecolor(CU['warm_white'])
fig.suptitle(f'Q3 - Reduced Form Model    $\lambda = \{\text{lam}\}$  | MATH GR 5320',
             fontsize=13, fontweight='bold', color=CU['navy'])

ax = axes[0]
ax.plot(t, survival(t), color=CU['navy'], lw=2)
ax.axvline(5, color=CU['gold'], lw=1.2, ls='--', label='t = 5')
ax.fill_between(t, survival(t), where=(t <= 5),
               color=CU['gold'], alpha=0.15, label=f'P( $\tau \leq 5$ ) = {pa:.4f}')
ax.set_title('Survival Curve    $s(t) = e^{-\lambda t}$ ')
ax.set_xlabel('Years')
ax.set_ylabel('s(t)')
ax.set_ylim(0, 1.02)
ax.legend(fontsize=9)
cu_stamp(ax)

ax = axes[1]
density = lam * np.exp(-lam * t)
ax.plot(t, density, color=CU['navy'], lw=2, label='p(t) =  $\lambda e^{-\lambda t}$ ')
mask_5 = t <= 5
mask_34 = (t >= 3) & (t <= 4)
ax.fill_between(t, density, where=mask_5, color=CU['blue'], alpha=0.20,
               label=f'P( $\tau \leq 5$ ) = {pa:.4f}')
ax.fill_between(t, density, where=mask_34, color=CU['gold'], alpha=0.55,
               label=f'P( $3 < \tau \leq 4$ ) = {pb:.4f}')
ax.set_title('Default Time Density   p(t)')
ax.set_xlabel('Years')
ax.set_ylabel('p(t)')
ax.legend(fontsize=9)
cu_stamp(ax)

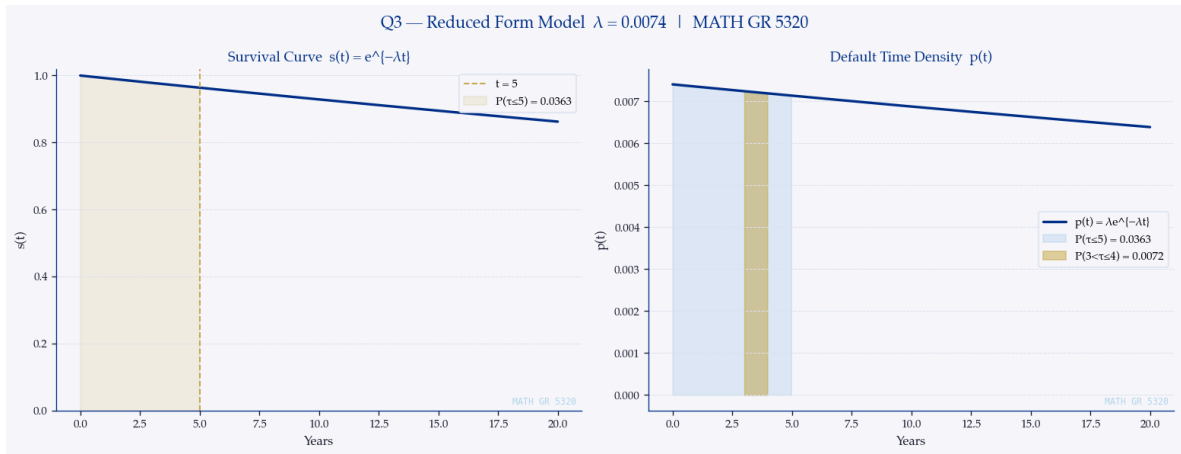
plt.tight_layout()
plt.show()

```

Q3 · Reduced Form Model $\lambda = 0.0074$

(a) $P(\text{default within 5 years}) = 0.036324$ (3.6324%)
 Formula: $1 - e^{(-\lambda \cdot 5)} = 1 - e^{(-0.0370)}$

(b) $P(\text{default between 3 and 4 years}) = 0.007211$ (0.7211%)
 Formula: $e^{(-\lambda \cdot 3)} - e^{(-\lambda \cdot 4)}$
 $= e^{(-0.0222)} - e^{(-0.0296)}$
 $= 0.978045 - 0.970834$



Reduced form modelling Mark II

```
In [79]: R_q4    = 0.30    # recovery rate
LGD_q4    = 1 - R_q4
r_q4      = 0.055    # risk-free rate

# Breakpoints and hazard values
# λ(t) = 0.010 for t ≤ 1.0
#       0.011 for 1.0 < t ≤ 2.0
#       0.012 for t > 2.0

def hazard(t):
    """Piecewise constant λ(t)."""
    t = np.atleast_1d(np.array(t, dtype=float))
    lam = np.where(t <= 1.0, 0.010,
                  np.where(t <= 2.0, 0.011, 0.012))
    return lam

def cumulative_hazard(t):
    """
    Λ(t) = ∫₀ᵗ λ(u) du - integrated piecewise.

    Piece 1 (t ≤ 1):    Λ(t) = 0.010 · t
    Piece 2 (1 < t ≤ 2): Λ(t) = 0.010 · 1 + 0.011 · (t-1)
    Piece 3 (t > 2):    Λ(t) = 0.010 · 1 + 0.011 · 1 + 0.012 · (t-2)
    """
    t = np.atleast_1d(np.array(t, dtype=float))
    L = np.where(
        t <= 1.0,
        0.010 * t,
        np.where(
            t <= 2.0,
            0.010 * 1 + 0.011 * (t - 1.0),
            0.010 * 1 + 0.011 * 1 + 0.012 * (t - 2.0)
        )
    )
```

```

    )
    return L

def survival_q4(t):
    """ $s(t) = e^{-\Lambda(t)}$ """
    return np.exp(-cumulative_hazard(t))

def density_q4(t):
    """ $p(t) = \lambda(t) \cdot s(t)$ """
    return hazard(t) * survival_q4(t)

def zcb_spread(t):
    """
    Spread  $S(t)$  for ZCB maturing at  $t$  (Lecture Slide 30):
     $v = e^{-rT} \cdot (1 - \text{LGD} \cdot (1 - s(T)))$ 
     $S = -1/T \cdot \log(1 - \text{LGD} \cdot (1 - s(T)))$ 
    """
    t = np.atleast_1d(np.array(t, dtype=float))
    s = survival_q4(t)
    inner = np.clip(1 - LGD_q4 * (1 - s), 1e-9, 1)
    return -np.log(inner) / t

print('=' * 60)
print('  Q4 · Piecewise Hazard Rate Model')
print('  R = 30%   LGD = 70%   r = 5.5%')
print('=' * 60)
print(f'  {"t":>5}  {"λ(t)":>8}  {"Λ(t)":>8}  {"s(t)":>8}  {"p(t)":>8}  {')
print('  ' + '-' * 58)
for t_val in [0.5, 1.0, 1.5, 2.0, 3.0, 5.0, 7.0, 10.0]:
    lv = hazard(t_val)[0]
    Lv = cumulative_hazard(t_val)[0]
    sv = survival_q4(t_val)[0]
    pv = density_q4(t_val)[0]
    sp = zcb_spread(t_val)[0] * 1e4
    print(f'  {t_val:>5.1f}  {lv:>8.4f}  {Lv:>8.5f}  {sv:>8.6f}  {pv:>8.6f}  {')

```

Q4 · Piecewise Hazard Rate Model
R = 30% LGD = 70% r = 5.5%

t	$\lambda(t)$	$\Lambda(t)$	$s(t)$	$p(t)$	Spread (bp)
0.5	0.0100	0.00500	0.995012	0.009950	69.95
1.0	0.0100	0.01000	0.990050	0.009900	69.89
1.5	0.0110	0.01550	0.984620	0.010831	72.16
2.0	0.0110	0.02100	0.979219	0.010771	73.27
3.0	0.0120	0.03300	0.967539	0.011610	76.62
5.0	0.0120	0.05700	0.944594	0.011335	79.11
7.0	0.0120	0.08100	0.922194	0.011066	80.01
10.0	0.0120	0.11700	0.889585	0.010675	80.44

```

In [80]: t_grid = np.linspace(0.001, 10, 1000)

fig, axes = plt.subplots(2, 2, figsize=(14, 9))
fig.patch.set_facecolor(CU['warm_white'])
fig.suptitle('Q4 - Piecewise Hazard Rate Model | MATH GR 5320 | R=30%
             fontsize=13, fontweight='bold', color=CU['navy'])

ax = axes[0, 0]
ax.step(t_grid, hazard(t_grid), color=CU['navy'], lw=2, where='post')

```

```

ax.axvline(1.0, color=CU['grid'], lw=1, ls=':')
ax.axvline(2.0, color=CU['grid'], lw=1, ls=':')
ax.set_title('Hazard Rate  $\lambda(t)$ ')
ax.set_xlabel('t (years)')
ax.set_ylabel('λ(t)')
ax.annotate('0.010', xy=(0.5, 0.010), xytext=(0.5, 0.0105),
            ha='center', fontsize=9, color=CU['slate'])
ax.annotate('0.011', xy=(1.5, 0.011), xytext=(1.5, 0.0115),
            ha='center', fontsize=9, color=CU['slate'])
ax.annotate('0.012', xy=(5.0, 0.012), xytext=(5.0, 0.0125),
            ha='center', fontsize=9, color=CU['slate'])
cu_stamp(ax)

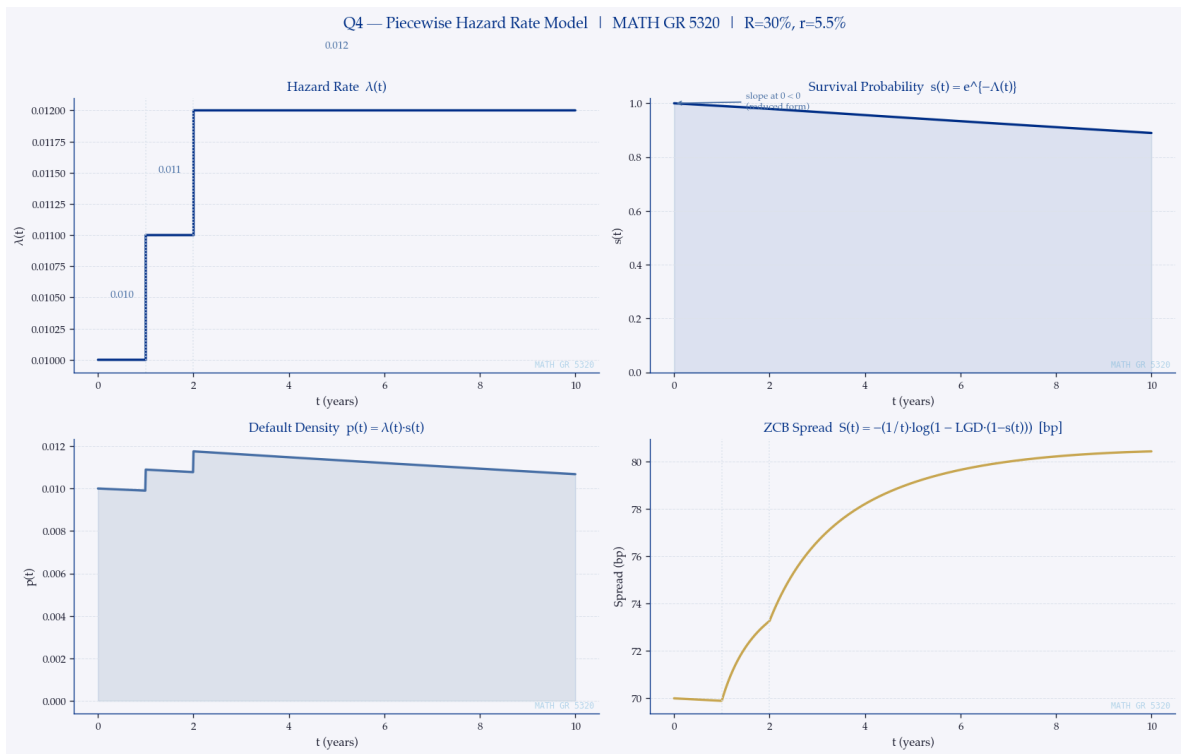
ax = axes[0, 1]
ax.plot(t_grid, survival_q4(t_grid), color=CU['navy'], lw=2)
ax.fill_between(t_grid, survival_q4(t_grid), alpha=0.10, color=CU['navy'])
ax.set_title('Survival Probability  $s(t) = e^{-\Lambda(t)}$ ')
ax.set_xlabel('t (years)')
ax.set_ylabel('s(t)')
ax.set_ylim(0, 1.02)
ax.annotate('slope at 0 < 0\n(reduced form)', xy=(0, 1),
            xytext=(1.5, 0.98), fontsize=8, color=CU['slate'],
            arrowprops=dict(arrowstyle='->', color=CU['slate']))
cu_stamp(ax)

ax = axes[1, 0]
ax.plot(t_grid, density_q4(t_grid), color=CU['slate'], lw=2)
ax.fill_between(t_grid, density_q4(t_grid), alpha=0.15, color=CU['slate'])
ax.set_title('Default Density  $p(t) = \lambda(t) \cdot s(t)$ ')
ax.set_xlabel('t (years)')
ax.set_ylabel('p(t)')
cu_stamp(ax)

ax = axes[1, 1]
spreads_bp = zcb_spread(t_grid) * 1e4
ax.plot(t_grid, spreads_bp, color=CU['gold'], lw=2)
ax.axvline(1.0, color=CU['grid'], lw=1, ls=':')
ax.axvline(2.0, color=CU['grid'], lw=1, ls=':')
ax.set_title('ZCB Spread  $S(t) = -(1/t) \cdot \log(1 - \text{LGD} \cdot (1-s(t)))$  [bp]')
ax.set_xlabel('t (years)')
ax.set_ylabel('Spread (bp)')
cu_stamp(ax)

plt.tight_layout()
plt.show()

```



```
In [81]: t1 = np.linspace(0, 1.0, 200)
t2 = np.linspace(1.0, 2.0, 200)
t3 = np.linspace(2.0, 10.0, 500)

fig, ax = plt.subplots(figsize=(13, 6))
fig.patch.set_facecolor(CU['warm_white'])

ax.axvspan(0, 1.0, alpha=0.06, color=CU['navy'], zorder=0)
ax.axvspan(1.0, 2.0, alpha=0.06, color=CU['gold'], zorder=0)
ax.axvspan(2.0, 10, alpha=0.06, color=CU['slate'], zorder=0)

ax.text(0.5, 1.0035, 'Piece 1\n $\lambda = 0.010$ ', ha='center', fontsize=8,
        color=CU['navy'], fontweight='bold')
ax.text(1.5, 1.0035, 'Piece 2\n $\lambda = 0.011$ ', ha='center', fontsize=8,
        color='#b8860b', fontweight='bold')
ax.text(6.0, 1.0035, 'Piece 3 -  $\lambda = 0.012$ ', ha='center', fontsize=8,
        color=CU['slate'], fontweight='bold')

ax.plot(t1, survival_q4(t1), color=CU['navy'], lw=3,
        label=r'Piece 1:  $s(t)=e^{-0.010t}$ ')
ax.plot(t2, survival_q4(t2), color='#c8a951', lw=3,
        label=r'Piece 2:  $s(t)=e^{-(0.010;+;0.011(t-1))}$ ')
ax.plot(t3, survival_q4(t3), color=CU['slate'], lw=3,
        label=r'Piece 3:  $s(t)=e^{-(0.021;+;0.012(t-2))}$ ')

for tv, label_str, xoff, yoff in [
    (1.0, f's(1) = {survival_q4(1.0)[0]:.4f}\n $\lambda(1) = 0.0100$ ', 0.15, -0.0),
    (2.0, f's(2) = {survival_q4(2.0)[0]:.4f}\n $\lambda(2) = 0.0210$ ', -0.8, -0.0)
]:
    sv = survival_q4(tv)[0]
    ax.plot(tv, sv, 'o', color=CU['gold'], ms=10, zorder=6,
            markedgcolor='white', markedgewidth=1.5)
    ax.annotate(label_str,
               xy=(tv, sv),
               xytext=(tv + xoff, sv + yoff),
               fontsize=8.5, color=CU['text'],
               bbox=dict(boxstyle='round,pad=0.3', fc='white',
```

```

        ec=CU['gold'], alpha=0.9),
        arrowprops=dict(arrowstyle='->', color=CU['gold'], lw=1.2

for tv, xoff in [(5.0, 0.15), (10.0, -1.1)]:
    sv = survival_q4(tv)[0]
    ax.plot(tv, sv, 's', color=CU['slate'], ms=7, zorder=5,
            markeredgecolor='white', markeredgewidth=1)
    ax.annotate(f's({tv:.0f}) = {sv:.4f}',
                xy=(tv, sv), xytext=(tv + xoff, sv - 0.008),
                fontsize=8, color=CU['slate'],
                bbox=dict(boxstyle='round,pad=0.2', fc='white',
                        ec=CU['slate'], alpha=0.8))

ax.annotate("slope at t=0:  $-\lambda(0) = -0.010$ \n(reduced form:  $s'(0) < 0$  always",
            xy=(0.05, survival_q4(0.05)[0]),
            xytext=(0.8, 0.998),
            fontsize=8.5, color=CU['navy'],
            bbox=dict(boxstyle='round,pad=0.3', fc='white',
                    ec=CU['navy'], alpha=0.9),
            arrowprops=dict(arrowstyle='->', color=CU['navy'], lw=1.2))

ax.text(0.5, 0.895, ' $\Lambda(t) = 0.010 \cdot t$ ',
        ha='center', fontsize=8, color=CU['navy'],
        bbox=dict(boxstyle='round', fc='white', ec=CU['navy'], alpha=0.7))
ax.text(1.5, 0.895, ' $\Lambda(t) = 0.010 + 0.011(t-1)$ ',
        ha='center', fontsize=8, color='#b8860b',
        bbox=dict(boxstyle='round', fc='white', ec='#c8a951', alpha=0.7))
ax.text(7.0, 0.895, ' $\Lambda(t) = 0.021 + 0.012(t-2)$ ',
        ha='center', fontsize=8, color=CU['slate'],
        bbox=dict(boxstyle='round', fc='white', ec=CU['slate'], alpha=0.7))

for xv in [1.0, 2.0]:
    ax.axvline(xv, color='gray', lw=1.2, ls='--', alpha=0.5)

ax.set_title('Q4 - Survival Function  $s(t) = e^{-\Lambda(t)}$  | '
            'Piecewise  $\lambda(t)$  | MATH GR 5320',
            fontsize=12, fontweight='bold', color=CU['navy'], pad=22)
ax.set_xlabel('t (years)', fontsize=10)
ax.set_ylabel('s(t)', fontsize=10)
ax.set_ylim(0.88, 1.010)
ax.set_xlim(0, 10)
ax.legend(fontsize=9, loc='lower left',
          framealpha=0.95, edgecolor=CU['grid'])
cu_stamp(ax)

plt.tight_layout()
plt.show()

```

